Zadanie č. 2: Stromy, stroje, hlasovania a redukcia dimenzie

- Dokumentácia

Meno:Martin Kasznár **Krúžok:piatok od 10:00 do 12:00**

Úvod

Zadanie súvisí s tým, že je nám predstavený dataset o dĺžke 8742 riadkov s 11 stĺpcami s rôznymi typmi hodnôt a jedným cieľovým stĺpcom s hodnotou počet požičaných bicyklov. Padlo rozhodnutie rozdeliť stĺpec **date** na **year**,**month**,**day,** zmazať duplicitný month a zmazať identifikátor instant, takže aj naďalej budeme hovoriť o 11 stĺpcoch reprezentujúcich 8742 rôznych analyzovaných hodín, na ktorých sú merané metriky a cieľový stĺpec predstavuje počet požičaných bicyklov v tieto hodiny. Ide o to spraviť odhadovací regresný nástroj koľko by v nevidenú hodinu, na ktorú by sme mali aspoň necieľové stĺpce pravdepodobne bolo požičaných bicyklov.
K tomu máme pristúpiť tak, že tento dataset ešte treba vyčistiť a potom naň treba po 1.) **zostaviť a predvádzať decision regression tree, sériu decision regression trees zapojenú v súborovom učení podľa výberu a support vector regression,** po 2.) **predviesť 1 jednoduchú 11D->3D EDA: redukcia podľa svojvôle a 1 jednoduchá PCA 11D->3D EDA,** po 3.) **zostaviť a predvádzať najlepší model z bodu 1 na datasete s manipulovanými príznakmi: je ich menej iba dobre korelujúce s cieľovým stĺpcom, je ich menej iba s vysokým feature importance skóre v ensemble a je ich menej PCA volí percento reprezentovanej variancie.**

0.0 edamap() - atomizovanie date, zľahčenie manipulácie s datasetom, redukcia príznakov

```
NOTHING = 'nic'
EDA_MAPPINGS = {
    'instant':     {np.nan: NOTHING},                                                                  #numer, free range
    'year':        {np.nan: NOTHING},                                                                  #numer, free range
    'month':       {np.nan: NOTHING, 'jan': 1, 'feb': 2, 'mar': 3, 'apr': 4, 'may': 5, 'jun': 6,
                    'jul': 7, 'aug': 8, 'sep': 9, 'oct': 10, 'nov': 11, 'dec': 12},                    #categ, cap4resolver
    'day':         {np.nan: NOTHING, **{f'd{i}': i for i in range(1, 32)}},                            #categ, cap4resolver
    'hour':        {np.nan: NOTHING, **{f'h{i}': i for i in range(0, 24)}},                            #categ, cap4resolver
    'holiday':     {np.nan: NOTHING},                                                                  #numer
    'weekday':     {np.nan: NOTHING, 'sun': 0, 'mon': 1, 'tue': 2, 'wed': 3, 'thu': 4, 'fri': 5, 'sat': 6}, #categ, cap4resolver
    'workingday':  {np.nan: NOTHING, 'False': 0, 'True': 1},                                           #numer
    'weather':     {np.nan: NOTHING, 'clear': 0, 'cloudy': 1, 'light rain/snow': 2, 'heavy rain/snow': 3},  #numer
    'temperature': {np.nan: NOTHING},                                                                  #numer, free range
    'humidity':    {np.nan: NOTHING},                                                                  #numer, free range
    'windspeed':   {np.nan: NOTHING},                                                                  #numer, free range
    'count':       {np.nan: NOTHING}                                                                   #numer, free range
}
KEPT_X_COLS = []
```

Pre ľahkosť navigácie množstva dát aj bez upozornenia a hlbšie v práci budú drvivú väčšinu času dáta namapované nasledovne: date je atomizovaný na year,month,day, prázdne miesto->NOTHING string konštanta, slová->čísla, plne automatizovane funkciou **edamap()** a v poslednom rade edamap bude vedieť, že si má nechať iba stĺpce v zozname KEPT_X_COLS a vždy stĺpec count, pričom „makro“ nechaj si všetky je prázdny KEPT_X_COLS. cap4resolver tag znamená, že niekedy by sme chceli iný popis pre onehot kódované stĺpce než je predvolený, ale onehot kódovanie je využité len pri veľmi špeciálnych testoch.

1.x mazanie id stĺpcov, null hodnôt, duplikátov, outlierov, zamyslenie nad kódovaním

- Načítajte dáta a pripravte ich na spracovanie modelmi ML - odstráňte stĺpce s identifikátormi, odstráňte null hodnoty, duplikáty, outliery, správne spracujte textové hodnoty (pozor: použite vhodné kódovanie, myslite pri tom na to, že spracovanie stĺpca, ktorý má príliš veľa unikátnych hodnôt môže pridať do vášho datasetu príliš veľa nových stĺpcov. Modely s ktorými budete robiť na tomto zadaní pracujú ťažšie s veľkým počtom stĺpcov). **1b**

1.1 cullid() - mazanie id stĺpca

Po prezretí štruktúry dát je zjavné, že ako identifikátor, ktorý je jedinečný pre dátový bod je schopný samostatne slúžiť iba stĺpec **instant** a pravdepodobne ani nenosí nejakú informáciu, z ktorej by bol odvoditeľný užitočný vzťah.

Tento stĺpec iba jednoducho zmažeme.

```
    0.0 Eda mapped : date->year,month,day; np.nan-> nic , words->numbers; cols kept:
    ['instant', 'year', 'month', 'day', 'hour', 'holiday', 'weekday', 'workingday',
    'weather', 'temperature', 'humidity', 'windspeed', 'count']
    1.1 Culled id col instant : (ENTIRE COL DELETED)
    OK
>>>
```

edamap()+cullid()

1.2 cullnull(null_toomany) - mazanie nullov

Po prezretí tabuľky v Exceli sa zistilo, že sa tu nachádza jedna hodnota nedeklarovaná v popise stĺpcov a tou je **prázdne miesto** a to vedieme ako NOTHING string konštantu. Keďže nemajú priamu informačnú hodnotu o analyzovanej hodine na to, za akých okolností sa požičalo koľko bicyklov, je 1. riešenie mazať riadky čo majú prázdne miesto, no to by nemuselo prejsť, ak by sme tým zmazali polovicu či 2/3 datasetu a tak sa model nenaučí nič. Padlo rozhodnutie funkciou **cullnull(hranica iného typu mazania)**, ak sa NOTHING vyskytuje pod stĺpcom málokrát mazať riadky, ak moc veľakrát, rovno odstrániť celý stĺpec a riadkami nemanipulovať. Takto sú dáta snáď výpovednejšie a hlavne o krok bližšie k úplnej numerizácii.

```
>>> df.isna().sum()
year          0
month         0
day           0
hour          0
holiday       0
weekday       0
workingday    0
weather       0
temperature   0
humidity      0
windspeed     0
count         0
dtype: int64
>>>
```

```
>>> df.isna().sum().sum()
0
>>> df.eq('nic').sum().sum()
0
>>>
```

'nic'<-prázdne miesto v

```
    0.0 Eda mapped : date->year,month,day; np.nan-> nic , words->numbers; cols kept:
    ['instant', 'year', 'month', 'day', 'hour', 'holiday', 'weekday', 'workingday',
    'weather', 'temperature', 'humidity', 'windspeed', 'count']
    1.1 Culled id col instant : (ENTIRE COL DELETED)
    1.2 Culled nulls under holiday : 11x nic rows removed
    OK
>>>
```

edamap()+cullid()+cullnull(2000)

1.3 culldupl() - mazanie duplikátov

Jednoducho zmažeme duplikáty.

```
    1.3 Culled duplicates : 7x duplicated rows removed
    OK
>>>
```

edamap()+cullid()+cullnull(2000)+culldupl()

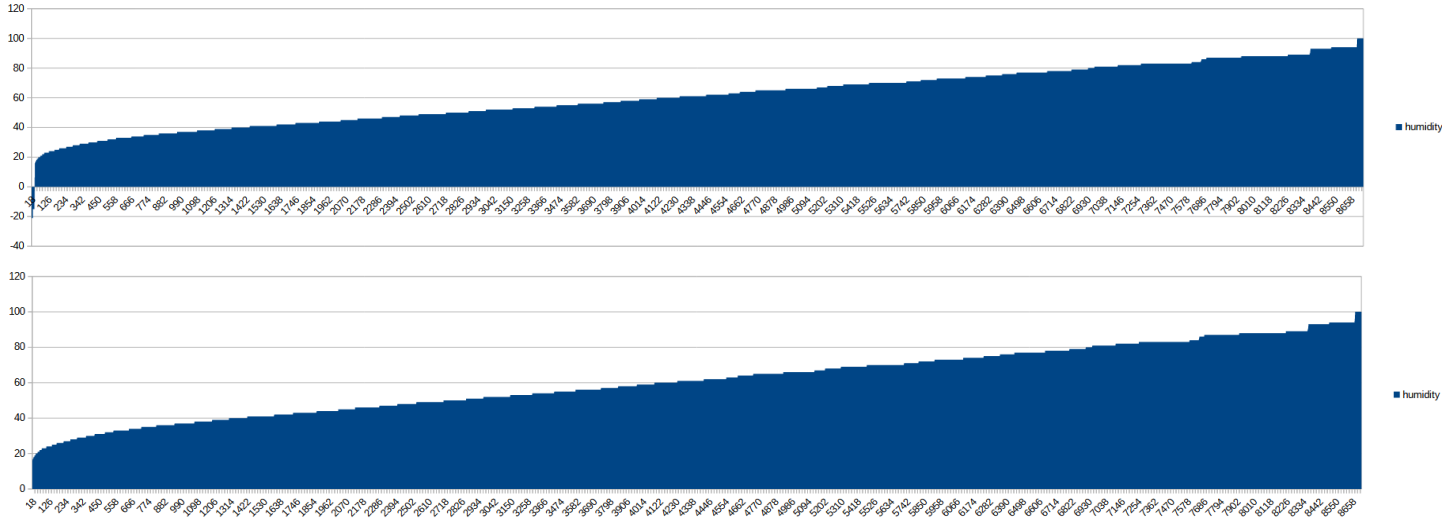
1.4 culloutl(outl_bounds) - mazanie outlierov

V prvom rade si vymenujme typy stĺpcov: máme numerické a kategorické aspoň pre túto tabuľku. V kategorických nejde zas o také rozsahy, keďže neskôr ich v zadaní mienime zmeniť na onehot encoding, čo je vlastne explózia jednej kategórie na mnoho s binárnym tvrdením či je to jedna z podkategórií kategórie. **V algoritmoch to ale nepoužijeme s výnimkou bivariantnej analýzy a PCA funkcionality, kde nenumerické<=cyklické ordinály sú NEANALYZOVATEĽNÉ pomocou PCA, keďže numerickosť'<=>odvoditeľné korelácie SÚ PODMIENKOU v oboch. Na tieto a iba tieto 2 use casey vynútime numerickosť stĺpcov onehot encodingom. Ide najmä o to, že decision trees vyžadujú nízky počet stĺpcov a cyklická ordinalita nie je likvidujúca prekážka**. Boli poskytnuté 3 rozsahy outl_bounds k numerickým stĺpcom s voľným rozsahom free range, kde jedine je dôležité mazať outlierov a nevadí, že 3 ďalším takýmto stĺpcom instant,year,count neboli poskytnuté, lebo instant sa v zadaní nepoužije,year sedí v každej hodnote na 2012 a count je cieľový stĺpec.
Princípom naivného outlier mazania podľa slovníka **culloutl({,a':[lo1,hi1],'b':[lo2,hi2],...})** je to, že hľadíme na to, pre každý numerický free range stĺpec aké číslo máme na pozícii nízkej hranice a na pozícii vysokej hranice. Snahou je zabezpečiť aby všetko vľavo od nízkej hranice a vpravo od vysokej hranice šlo z dataframeu preč (riadok).

```
1.4 Culled outliers under temperature :  0 x rows removed
1.4 Culled outliers under humidity :  22 x rows removed
1.4 Culled outliers under windspeed :  0 x rows removed
OK
>>>
```

edamap()+cullid()+cullnull(2000)+culldupl()+
+culloutl({'temperature': [-40, 40], 'humidity': [0, 100], 'windspeed': [0, 110]})

Demonštrujme prakticky:



Prevádzame na stĺpcoch ako si rozkáže vstup do funkcie culloutl:

```
culloutl({'temperature': [-40, 40], 'humidity': [0, 100], 'windspeed': [0, 110]})
```

(BIVAR/PCA.ONLY) dummify() - onehot encoding

V projekte nastáva voči decision trees nelikvidačný problém **kategorických stĺpcov** a tu ide o to, že dočasné číselné mapovania napr. weekday 0-nedeľa a 1-pondelok si absolútne nie sú blízke v skladbe okolností analyzovanej hodiny, kým pri temperature ako numerickom type stĺpca by tá skladba okolností mohla byť podobná. Toto sa stáva menším problémom, keď pod stĺpcom, pod ktorým môže byť viac možností nie sú zároveň ani dve protipólne vzájomne opačné. Pre decision trees to **nie je až tak likvidačné keď v kategorických stĺpcoch máme blízkosť hodnôt identifikátora vlastnosti 1 a 2, ktoré si akostne nie sú takisto blízke** ako by bolo **totálne likvidačné do decision trees zakomponovať 50-60 stĺpcov ak by sme podstúpili onehot encoding**. V bočných scenároch ako bivariantná analýza či PCA funkcionalita onehot encoding ale musíme použiť a iba tu. Podľa princípu onehot encoding iba tu explodujeme 1-12 na 12 rôznych stĺpcov s hodnotami 0 či 1 funkciou **dummify()**.

continent	labelEncoding	Africa	Asia	Europe	North America	Oceania	South America
Europe	2	0	0	1	0	0	0
Africa	0	1	0	0	0	0	0
Africa	0	1	0	0	0	0	0
Africa	0	1	0	0	0	0	0
South America	5	0	0	0	0	0	1

```
(BIVAR/PCA.ONLY) Performed dummification: ['month', 'day', 'hour', 'weekday'] dummified
OK
>>>
```

edamap()+cullid()+cullnull(2000)+culldupl()+
+culloutl({'temperature': [-40, 40], 'humidity': [0, 100], 'windspeed': [0, 110]})+
dummify()

Demonštrujme:

month
12
11
8
5
10
12
7
2
11

jan	feb	mar	apr	may	jun	jul	aug	sep	oct	nov	dec
0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0

Prevádzame na týchto stĺpcoch:

```
CATEG_X_COLS = ['month', 'day', 'hour', 'weekday']
```

2 tvorba x/y množín, následné delenie na train/test

- Rozdeľte dáta na trénovaciu a testovaciu množinu (validačnú Vám v tomto zadaní netreba), následne na vstupnú a výstupnú množinu, potom normalizujte (výstup nenormalizujete).

x a **y** množina sa rodí tak, že prečítame tabuľku knižnicou pandas, čo ju nahodí do dataframeu df, potom na nej robíme čistenie dát (to je popísané vyššie), následne zostavíme hlbokú kópiu df a count (count lebo je cieľ) preloženú do numpy.ndarray, toto nazývame **x** a hlbokú kópiu stĺpca df count preloženú do numpy.ndarray, toto nazývame **y**. **Motivácia je naučiť model podľa x, čo máme na analyzovanú hodinu, predpovedať y.**
Vyberme si pomer rozdelenia dát 8:2. tn, tt nám slúžia na výber, stačí napísať za rovná sa 8,2 a toto programu cez train_test_split(…) povie, že trénovacej množine dajme 80% a 20% ide testovacej množine. **Motivácia delenia je v tom, že musíme mať čím natrénovať decision tree, snáď dostatočne všeobecne na nevidenú hodinu. Testovacia množina slúži na jediné**

otestovanie a toto má simulovať náhodnú analyzovanú hodinu, ktorá už v produkcii začala alebo má taká byť a chceme otestovať či bude podľa metrík pravdepodobne finančne úspešná, lebo testovacia množina v trénovaní nie je videná modelom a my skúšame či je model dostatočne všeobecný.

```
#-init--------------------------------------
df = pd.read_csv('z2_data_1y.csv', sep=';')
pd.set_option('future.no_silent_downcasting', True)
pd.set_option('display.max_columns',None)
pd.set_option('display.width',None)
silent = 0
do_3,do_41,do_42 = 0,0,0
do_univar,do_bivar = 0,0
```

~~dataset cleaning~~

```
#2
x = copy.deepcopy(df.drop(columns=['count']).to_numpy())
y = copy.deepcopy(df['count'].to_numpy())

tn, tt = 8,2
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=(tt)/(tn+tt), random_state=42
)
```

~~working with data~~

```
#-export-----------------
x=np.concatenate((x_train, x_test))
y=np.concatenate((y_train, y_test))
new_dfxfrag = pd.DataFrame(x, columns=df.drop(columns=['count']).columns)
new_dfyfrag = pd.Series(y, name='count')
df = pd.concat([new_dfxfrag, new_dfyfrag], axis=1)
df.to_csv('out.csv', sep=';', index=False)
print('OK')
```

```
>>> len(x_train)
6960
>>> len(x_test)
1741
>>> len(x)
8701
>>>
```

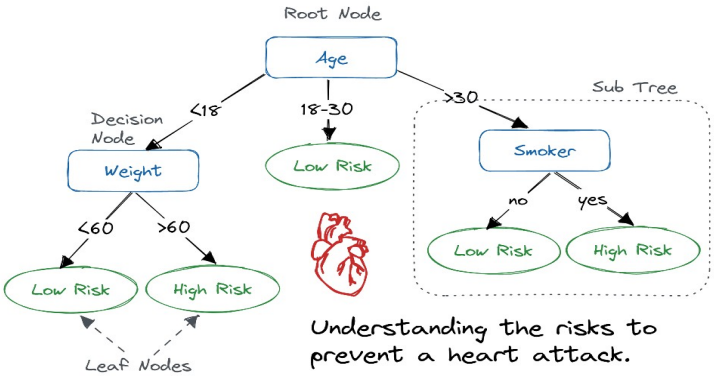
3.x Rozhodovací strom, zobraziteľný slabší strom, ensemble model, predvedenie obsluhy ensemble modelu, model SVM, na všetkom MSE, R2, reziduály

- Trénujte nasledujúce modely (pre každý model dosiahnite kladné R2 skóre[1] - pri najlepšom modeli aspoň 0.7):
  - rozhodovací strom **0.5b**:
    - jeden strom z výsledkov aj zobrazte do dokumentácie - ak budú Vaše stromy príliš veľké, pre vizualizáciu natrénujte menší strom, aj keby mal mať horšie výsledky, je potrebné ho vedieť analyzovať; **0.5b**
  - Vami vybraný stromový súborový (*ensemble*) model **0.5b**:
    - vizualizujte dôležitosť vstupných parametrov - ak ich bude príliš veľa, zredukujte ich na podmnožinu najdôležitejších; **0.5b**
  - model SVM. **1b**

  Modely vyhodnoťte na trénovacej a testovacej množine pomocou MSE (príp. RMSE), R2 a výsledky vizualizujte tak, aby ste mohli analyzovať reziduály (pozor: treba vizualizovať reziduály, tj. **nie** očakávanú hodnotu vs. predpovedanú hodnotu). Navzájom porovnajte modely. **1b**

3.1 Rozhodovací strom, zobraziteľný slabší strom + metriky

V takomto strome ide o to, že **DecisionTreeRegressor** ako bude ukázané na obrázku automaticky delí do odnoží či v rozsahu rôznej premennej a v tomto rozsahu v ďalších a ďalších rozsah rôznych premenných vnorene koľko v tomto bode tipnúť cieľový stĺpec, že má hodnotu. Otestujeme tak, že oproti továrenskemu nastaveniu programu prepíšeme **do_3=1** a pod if do_3 na čitateľný rozhodovací strom voľme **_3(x_train, x_test, y_train, y_test, x, y, 8, 30, 340, 15)**, kde takto povieme v prvých dvoch premenných za y_test, že chceme rozhodovací **strom o hĺbke x s minimálne y listami** než dovolíme štiepenie ďalej do hlbších úrovní. **3, 1 analyzovateľné zobrazenie, 8, 1 by dalo ale lepšiu výkonnosť stromu.**



Trénovacia fáza by teda v programe spočívala v načítaní x_train,y_train a testovacia fáza je spomínaný jeden test akoby z produkcie.

```
if do_3 or do_42:
    scaler = MinMaxScaler()
    x_train = scaler.fit_transform(x_train)
    x_test  = scaler.transform(x_test)
if do_3:
    _3(x_train, x_test, y_train, y_test, 8, 1, 8, 30, 340, 15)
```

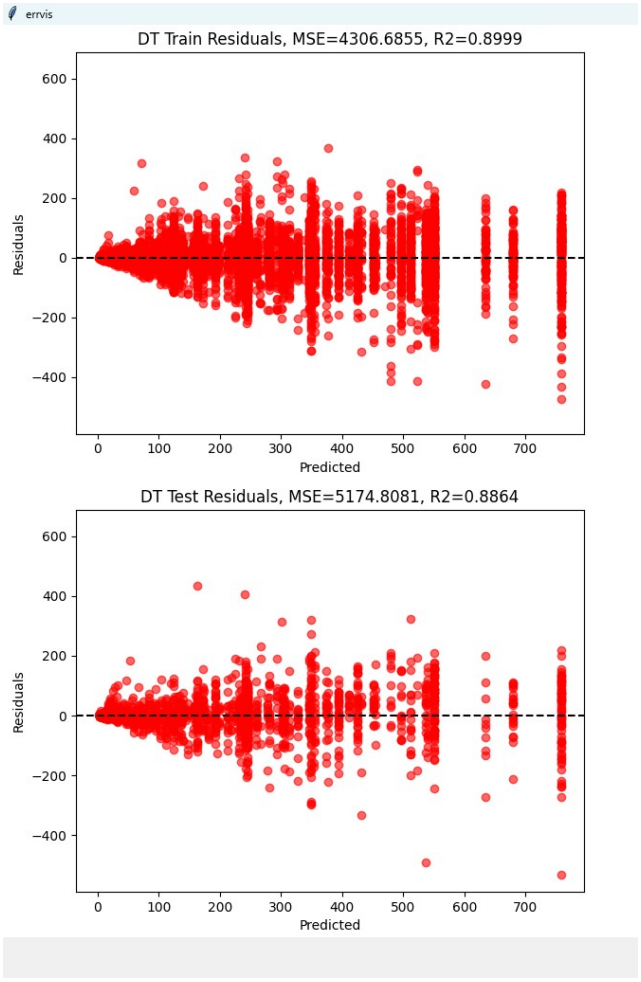
```
def _3(x_train, x_test, y_train, y_test, dt_mxd, dt_mnsl, ens_mxd, ens_ne, svm_c, svm_g):
    #3.1
    dt = DecisionTreeRegressor(max_depth=dt_mxd, min_samples_leaf=dt_mnsl, random_state=42)
    dt.fit(x_train, y_train)
    y_dt_train_pred = dt.predict(x_train)
    y_dt_test_pred = dt.predict(x_test)

    dtdl(dt, dt_mxd)
```

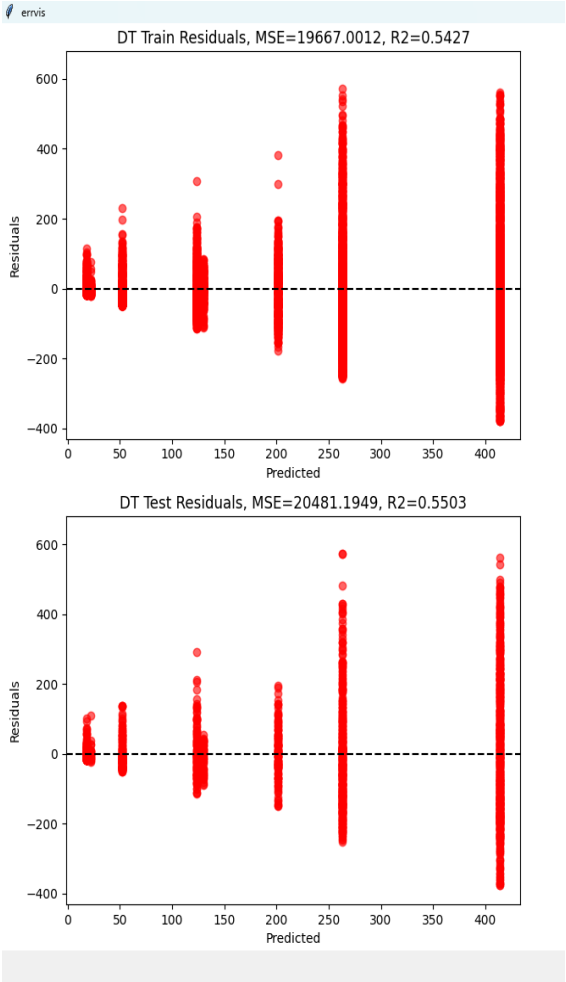
ako vypadá spustenie programu v širšej perspektíve

vytvorenie modelu->nasadenie modelu->predikovanie výsledku pre train mn.->test mn.->stiahnutie hotového jpg stromu do adresára skriptu

Výkonný vs. nevýkonný strom:



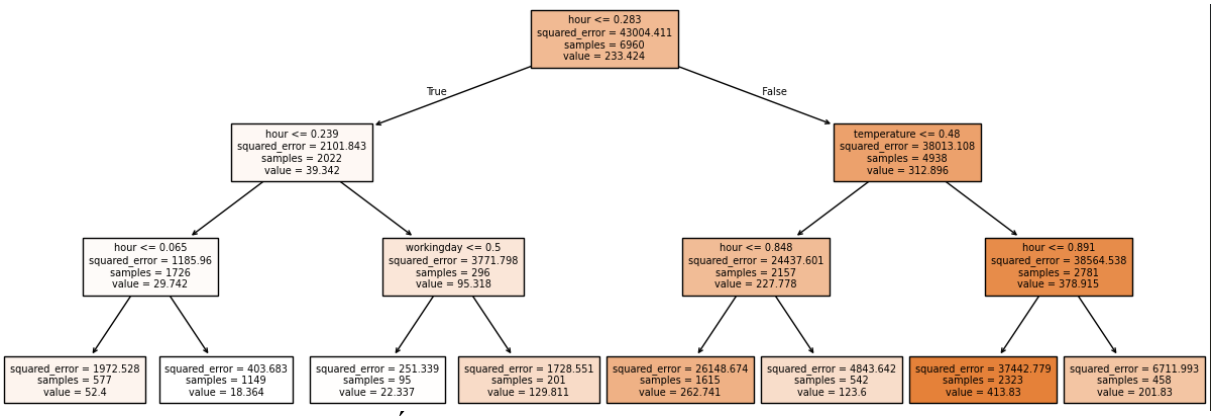
hĺbka 8, min. počet listov pred št. 1



hĺbka 3, min. počet listov pred št. 1

Konštatujeme, že vyššie uvedené testovacie R2 boli 0.886 a 0.550, čo značí, že hlbší strom v prípade, že sa vyhneme pretrénovaniu prehnanou hĺbkou odhaduje lepšie.

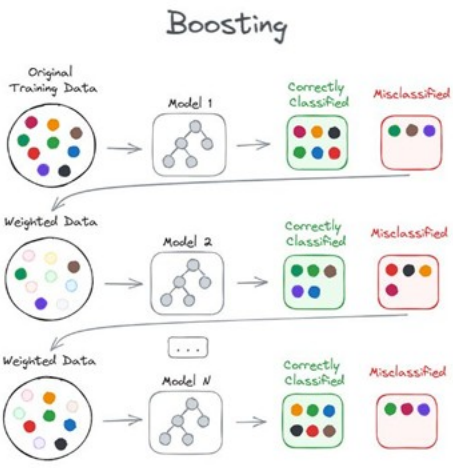
Zobraziteľný slabší strom



hĺbka 3, min. počet listov pred št. 1

na obrázku vidíme, že strom považuje hour, temperature, workingday za veľmi dôležité, takže ich stavia blízko ku koreňu a takisto je tu štruktúra, že ak by sme vedeli, že ubehlo <= 28% spozorovaných hodín dňa, tak bude požičaných 39 bicyklov, ak vyše, tak 312, potom na toto vieme baliť ďalšie podmienky využívajúc nielen stĺpec hour, napr. najviac 413 bicyklov požičiame, ak ubehlo >28% spozorovaných hodín dňa (>6.5h), teplota je >48% na polceste medzi min. a max. spozorovanou teplotou (>15.03C) a ubehlo <=89% spozorovaných hodín dňa (<=20.5h), takže 6.5h-20.5h ak je >15.03C môžu byť otváracie hodiny, hoci strom o hĺbke 3 veľmi zaostáva za stromom o hĺbke 8

3.2 ensemble model, predvedenie obsluhy ensemble modelu + metriky



Vybrali sme si metódu súborového učenia **boosting**. Boosting spočíva v tom, že najprv normálne zhotovíme decision tree v 1. behu a všimneme si ktoré vzorky už strom odhaduje dobre a nasledovne v 2. behu dolaďujeme už existujúci strom jemne: na odhadovanie veľmi zle v predošlom behu odhadnutých vzoriek, že aký bude cieľový stĺpec. Podľa toho koľko je **n_estimators** toľko behov. A v neposlednom rade je potrebné zadať hĺbku **max_depth**, ktorá charakterizuje hĺbku jednotlivých stromov jedného behu.

Na obrázku sú zobrazené pokročilé možnosti **AdaBoostRegressoru**, ktorý chceme použiť:

Parameters:

estimator : *object, default=None*

The base estimator from which the boosted ensemble is built. If `None`, then the base estimator is `DecisionTreeRegressor` initialized with `max_depth=3`.

Added in version 1.2: `base_estimator` was renamed to `estimator`.

n_estimators : *int, default=50*

The maximum number of estimators at which boosting is terminated. In case of perfect fit, the learning procedure is stopped early. Values must be in the range `[1, inf)`.

learning_rate : *float, default=1.0*

Weight applied to each regressor at each boosting iteration. A higher learning rate increases the contribution of each regressor. There is a trade-off between the `learning_rate` and `n_estimators` parameters. Values must be in the range `(0.0, inf)`.

loss : *{'linear', 'square', 'exponential'}, default='linear'*

The loss function to use when updating the weights after each boosting iteration.

random_state : *int, RandomState instance or None, default=None*

Controls the random seed given at each `estimator` at each boosting iteration. Thus, it is only used when `estimator` exposes a `random_state`. In addition, it controls the bootstrap of the weights used to train the `estimator` at each boosting iteration. Pass an int for reproducible output across multiple function calls. See Glossary.

estimator: vyberieme si napríklad rozhodovací strom na zapojenie do súborového učenia

n_estimators: počet behov

learning_rate: riskantnejšie učenie

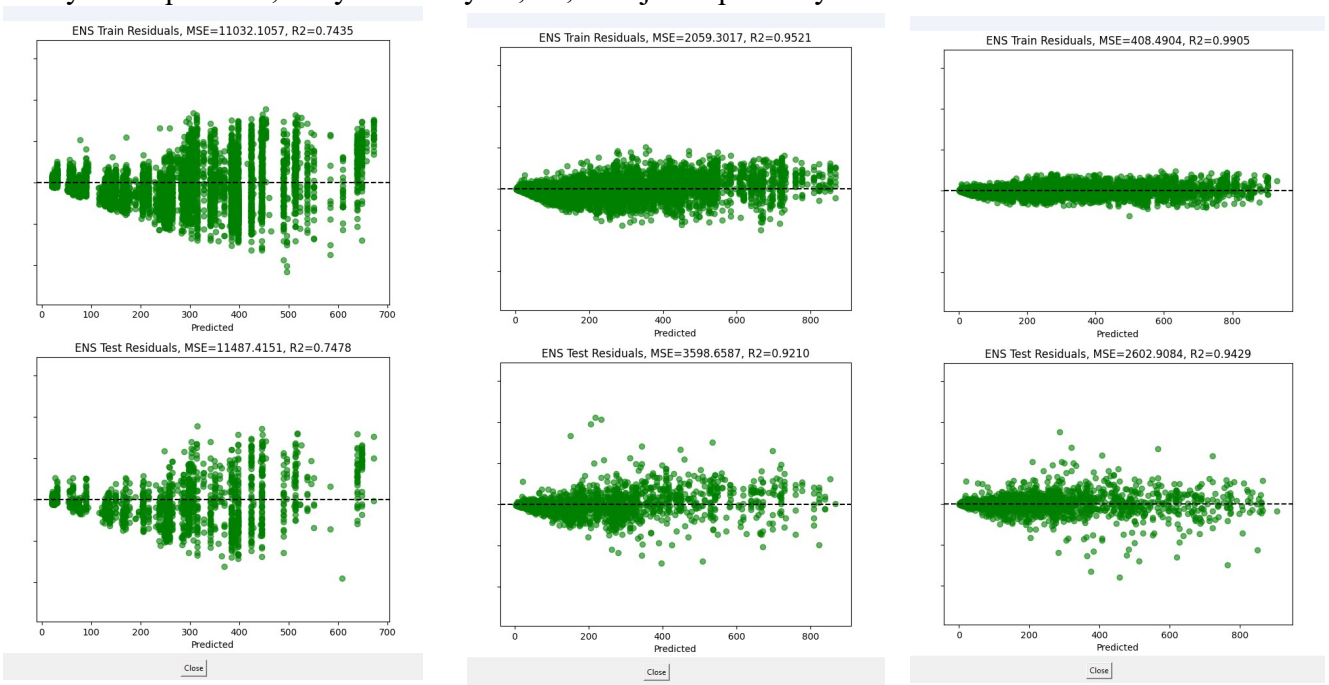
loss: rôzne loss funkcie pracujú lepšie s rôznymi typmi predpokladaných strát, napr. square by pracovalo lepšie s normálne rozdelenými dátami

random_state: reproducibilita pseudonáhodných rozhodnutí programom

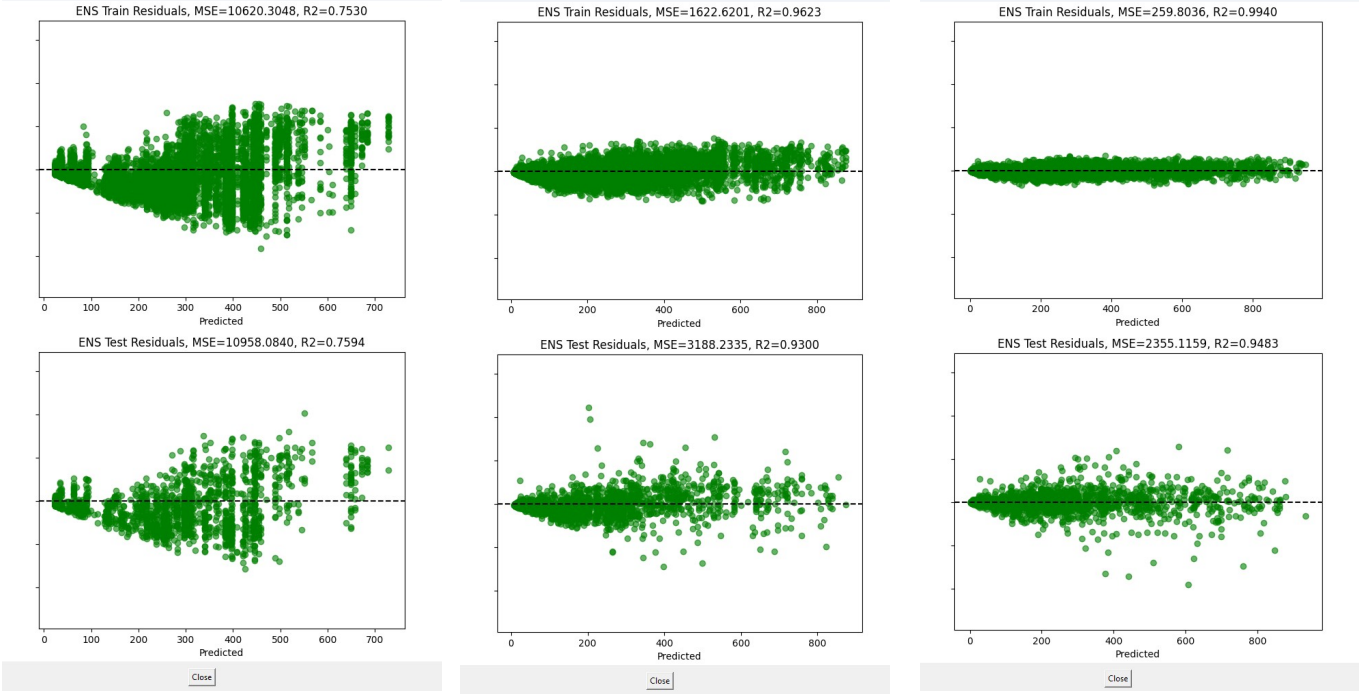
-

Predvedenie obsluhy ensemble modelu

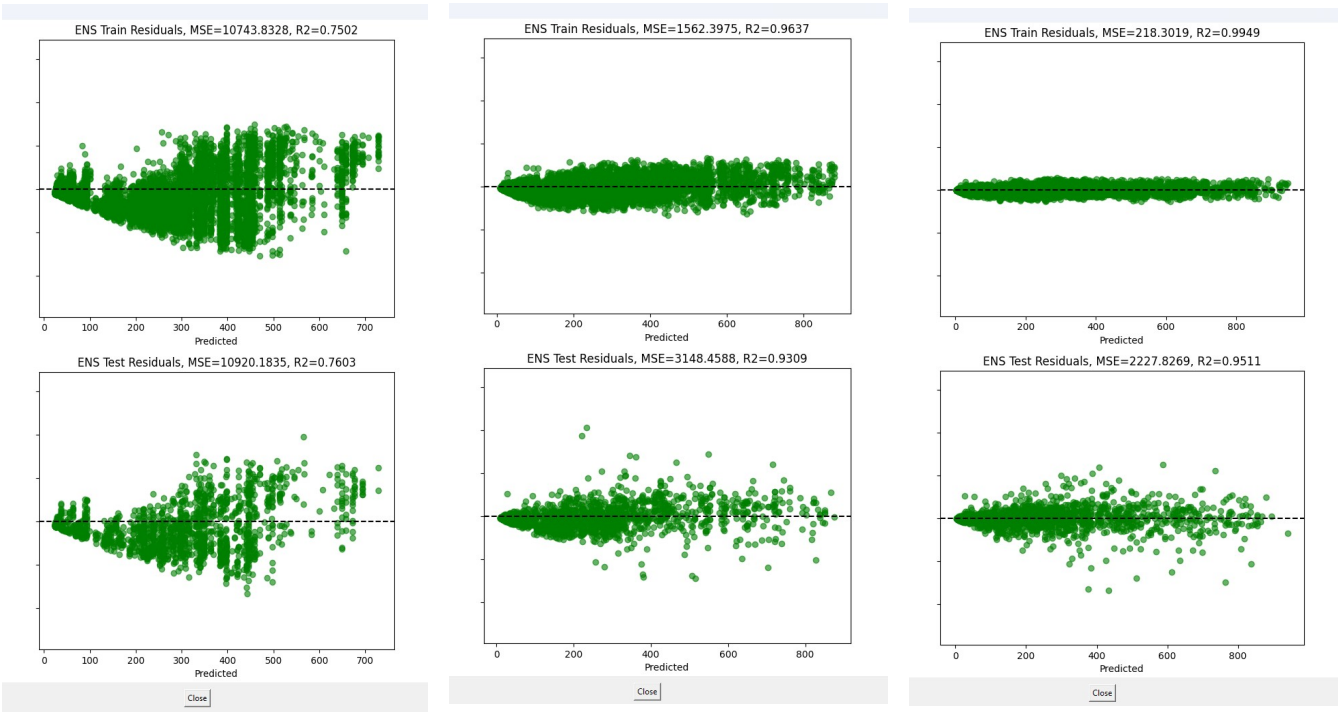
Podobne ako vyššie udajme **do_3=1** a pod if do_3 udajme **_3(x_train, x_test, y_train, y_test, 8, 1, z, w, 340, 15)**, pričom z, w značí **max_depth DT, n_estimators ENS.** Solídne rýchle a presné z, w by mohlo byť 8, 30, testujme s podobnými hodnotami v okolí:



zľava doprava: hĺbka=4, behov=15; hĺbka=8, behov=15; hĺbka=12, behov=15



zľava doprava: hĺbka=4, behov=30; hĺbka=8, behov=30; hĺbka=12, behov=30



zľava doprava: hĺbka=4, behov=45; hĺbka=8, behov=45; hĺbka=12, behov=45

Konštatujeme, že testovacie R2 boli nasledovné, pričom **vpravo dole sú najvyššie hĺbky a najväčší počet behov** a **vľavo hore najnižšie hĺbky a najnižší počet behov**:

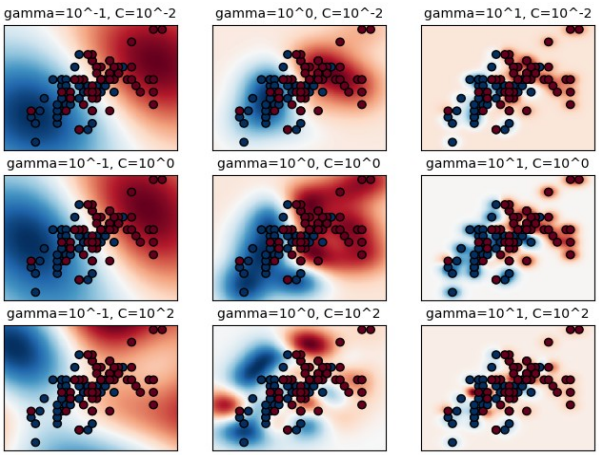
0.747->0.921->0.942

0.759->0.930->0.948

0.760->0.930->0.951

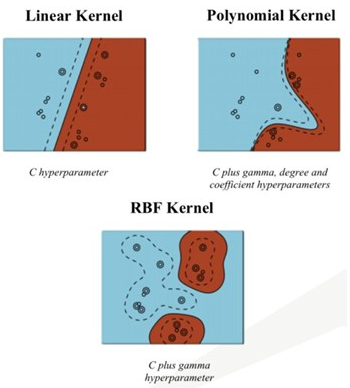
Z tohto vyplýva, že počet behov má strednú dôležitosť, keďže riadky sú si podobné, ale hĺbka extrémnu, keďže stĺpce si podobné nie sú.

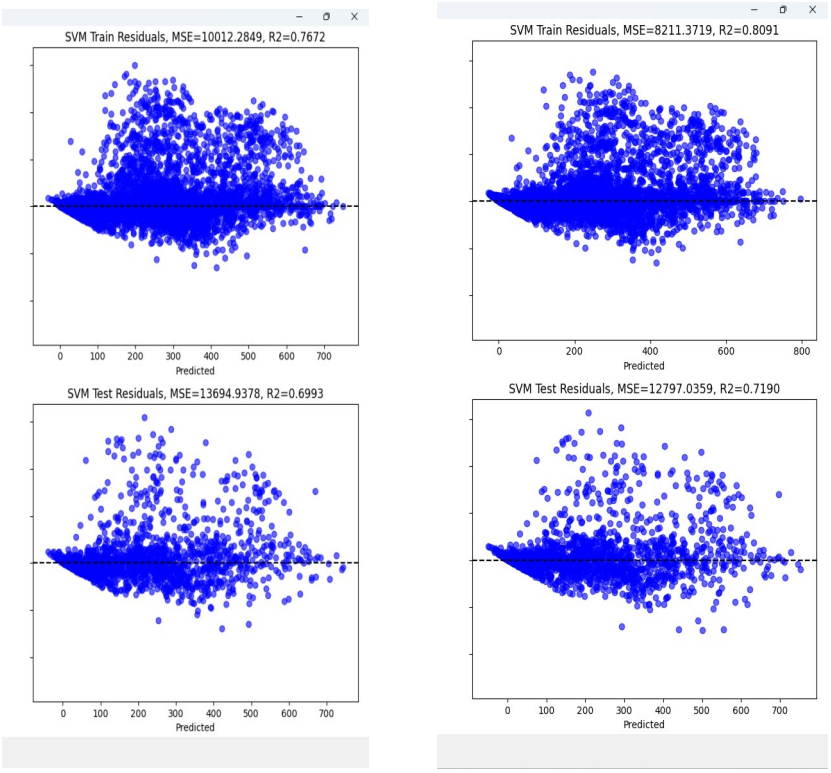
3.3 model SVM



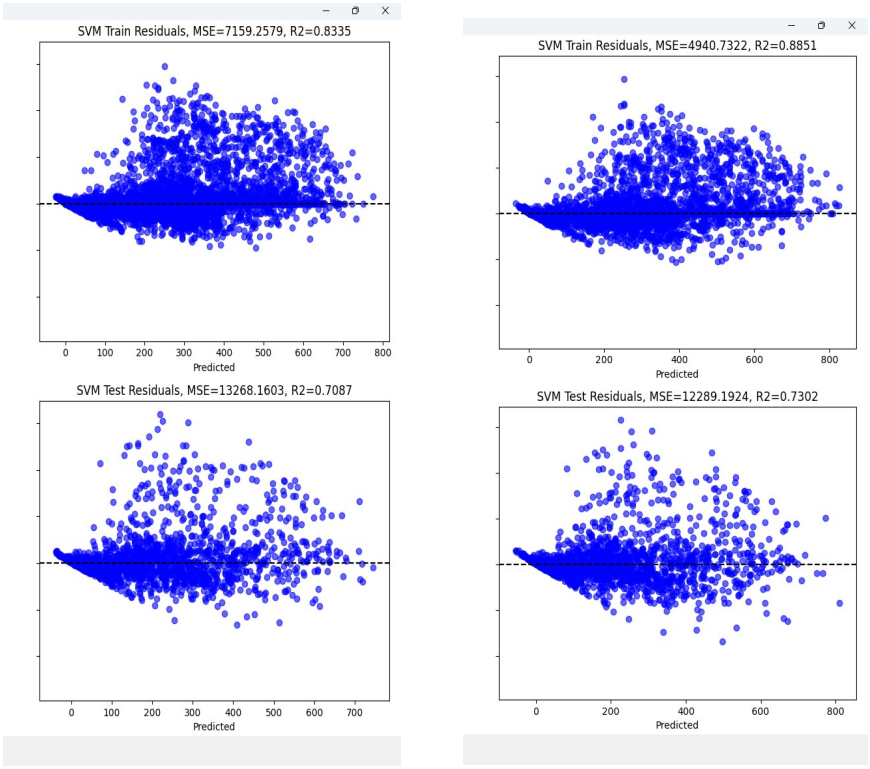
Support Vector Machine SVR (regressor) spočíva v tom, že sa snažíme ohraničiť hyperroviny okolo bodov, že koľko by im malo prislúchať množstva cieľového stĺpca. Padlo rozhodnutie použiť **Kernel Radial Basis Function**, keďže sa počas testovania jediný javil jednoducho zostaviteľný na približovanie k rozumnému R2 skóre. Ovládame to cez **_3(x_train, x_test, y_train, y_test, 8, 1, 8, 30, v, u)**, kde v, u značí **C regularizačný parameter, gamma kernelový** koeficient. C umožňuje komplikovanejší tvar, gamma súvisí s veľkosťou. Otestované v, u sú 340, 15 a tie majú adekvátne výsledky. Preveďme aspoň menší počet testov:

pozor, toto je iba ilustračný obrázok!^





zľava doprava: C=170, gamma=7.5; C=340, gamma=7.5



zľava doprava: C=170, gamma=15; C=340, gamma=15

Konštatujeme, že testovacie R2 boli nasledovné, pričom **vpravo dole sú najvyššie hĺbky a najväčší počet behov** a **vľavo hore najnižšie hĺbky a najnižší počet behov**:

0.699->0.719

0.708->0.730

Z tohto vyplýva, že C má strednú dôležitosť, keďže riadky sú si podobné, ale aj gamma strednú, keďže stĺpce sú si podobné tiež. Z tohto zatiaľ nevyplýva záver.

Víťazný model

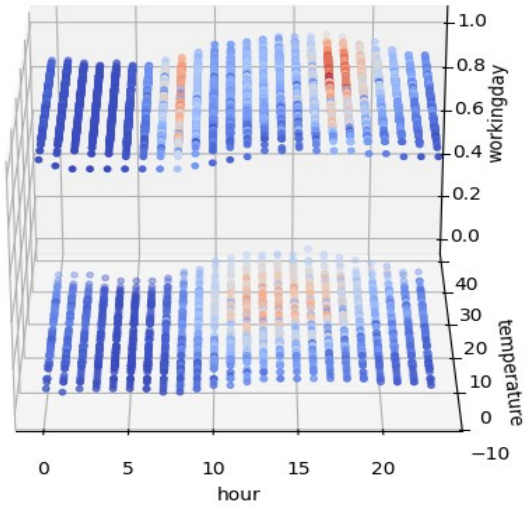
S R2 skóre **0.951**vyhráva **ensemble model hĺbka=12, behov=45** a znamená to to, že v 5. časti úlohy nebudeme spúšťať pôvodný návrh: _3(x_train, x_test, y_train, y_test, 8, 1, 8, 30, 340, 15), ale **_3(x_train, x_test, y_train, y_test, 8, 1, 12, 45, 340, 15)** aj **_42(x_train, x_test, y_train, y_test, 8, 1, 12, 45, 340, 15)**

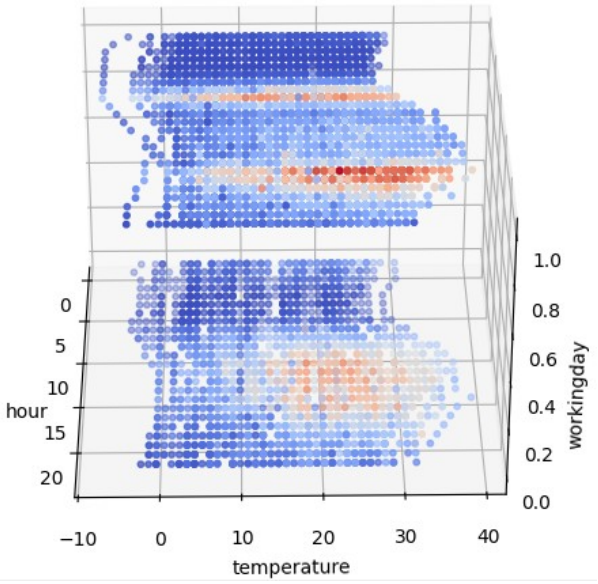
4.1 jednoduchá EDA na 11D->3D datasete: redukcia podľa svojvôle

- Sledujte, čo s dátami spraví redukcia dimenzie (pomocou 3D bodových grafov - scatter plot):
  - Vyberte si 3 príznaky (pred normalizáciou), ktoré budú na osiach. Snažte sa nájsť také príznaky, pri ktorých budete vedieť graf analyzovať. Dáta vyfarbite podľa výstupného parametra (count). Pokúste sa z grafu vyčítať nejakú závislosť. **1b**

ako zaujímavé z hľadiska analýzy sa javia **hour**, **temperature**, **workingday**, keďže v 3.1 si ich vybral zobraziteľný slabší strom ako prvé 3, čo znamená, že sú značne informatívne

Vypneme **do_3=0**, zadať **KEPT_X_COLS = ['hour','temperature','workingday']**, zapnúť **do_41=1** a pod if do_41 skontrolovať príkaz **_41('hour','temperature','workingday')**





Z grafu po zamyslení vyplýva, že ľudia si cez pracovné dni, **keď idú do práce** okolo 8:00 **či z práce** 17:00/18:00 **hromadne požičiavajú bicykle**, kým keď sa nepracuje sa bicyklujú rovnomernejšie počas svetla od 11:00 do 17:00.

Čo sa týka teploty, tak graf jasne ukazuje, že ľudia si požičajú bicykel **do práce či na cestu z práce** aj za **veľmi nízkej teploty**, kým keď sa nepracuje si na bicykli vyjdú len ak je teplo.

Posledný postreh je, že **v pracovné dni si počas pracovnej doby nemá kto požičiať bicykle**.

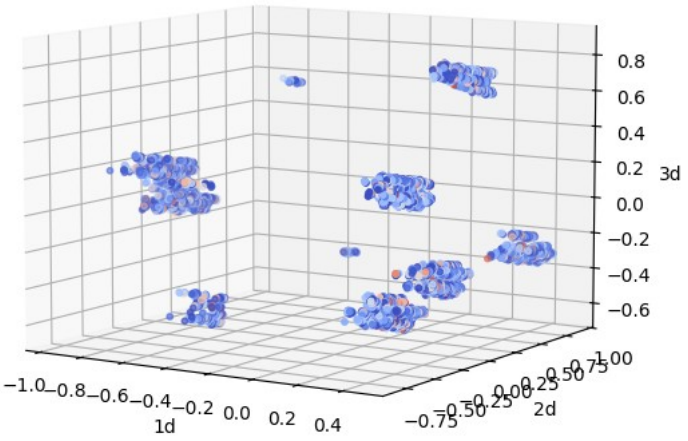
Bolo by dobré požičiavať bicykle za čo najtmavočervenejších okolností.

4.2 jednoduchá EDA na PCA 11D->3D datasete a porovnanie s 4.1

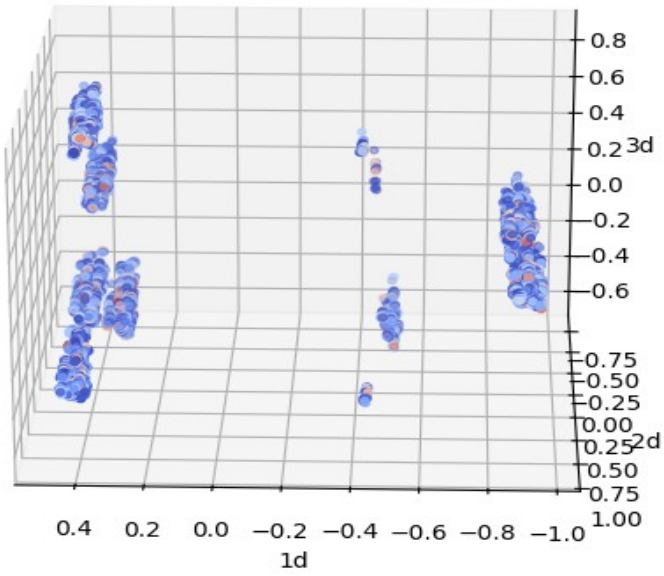
– Minimalizujte množinu (po normalizácii, bez výstupného parametra) na 3 dimenzie (pomocou PCA, UMAP ...), tie vyneste na osi, dáta opäť zafarbite podľa výstupného parametra (count). **1b**

Grafy navzájom porovnajte.

Vypneme **do_41=0**, zadať **KEPT_X_COLS = [],** zapnúť **do_42=1** a pod if do_42 skontrolovať príkaz **_42(x_train, x_test, y_train, y_test, 8, 1, 12, 45, 340, 15)**



Základný princíp PCA spočíva v tom, že našich 11 rozmerov má väčšinou neefektívnu varianciu vzhľadom na to, že môže viac stĺpcov korelovať nielen s cieľom, ale aj necieľom a PCA sa snaží natlačiť čo najviac informácií do čo najmenej rozmerov, pričom si nasledovne vyberáme počet zobrazených rozmerov.



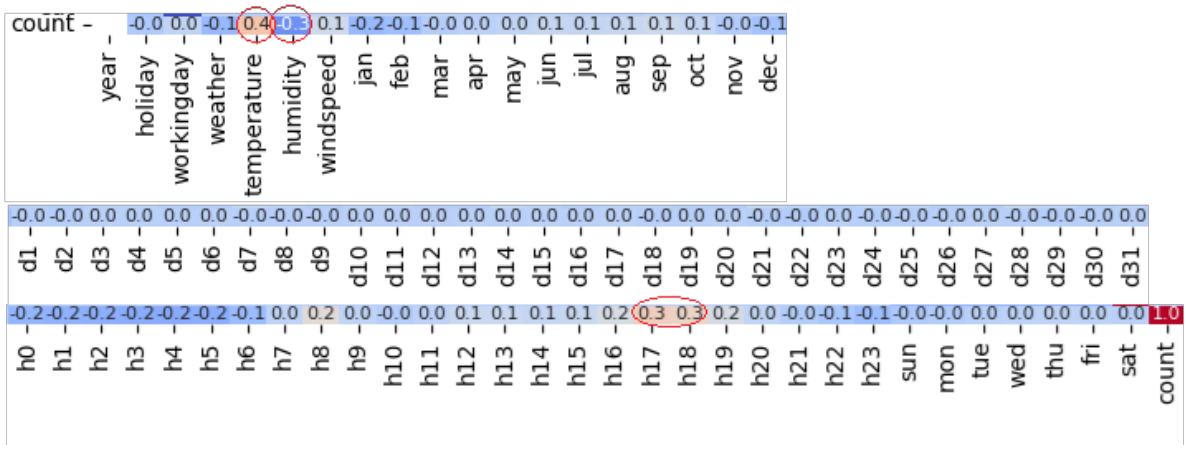
Keďže vidíme jednoznačne oddelené 2 skupiny dát v tvare ponoriek od 5 skupín dát v podobnom tvare, workingday je dobrý príznak, týždeň má 7 dní a soboty, nedele nie sú pracovné dni, môže toto PCA napovedať, že ľudia sa správajú cyklicky rovnako podľa toho aký je deň v týždni? (pon/ut/…) s ohľadom na požičiavanie bicyklov.

PCA 11D->3D graf nie je ľahko čitateľný narozdiel od svojvoľne redukovaného do 3D grafu, keďže je plný aj zlých závislostí, pričom svojvoľne redukovaný má vybrané iba tie najinformatívnejšie.

5.1 Víťazný model rieši množinu iba s kvalitnými príznakmi podľa korelačnej matice + metriky

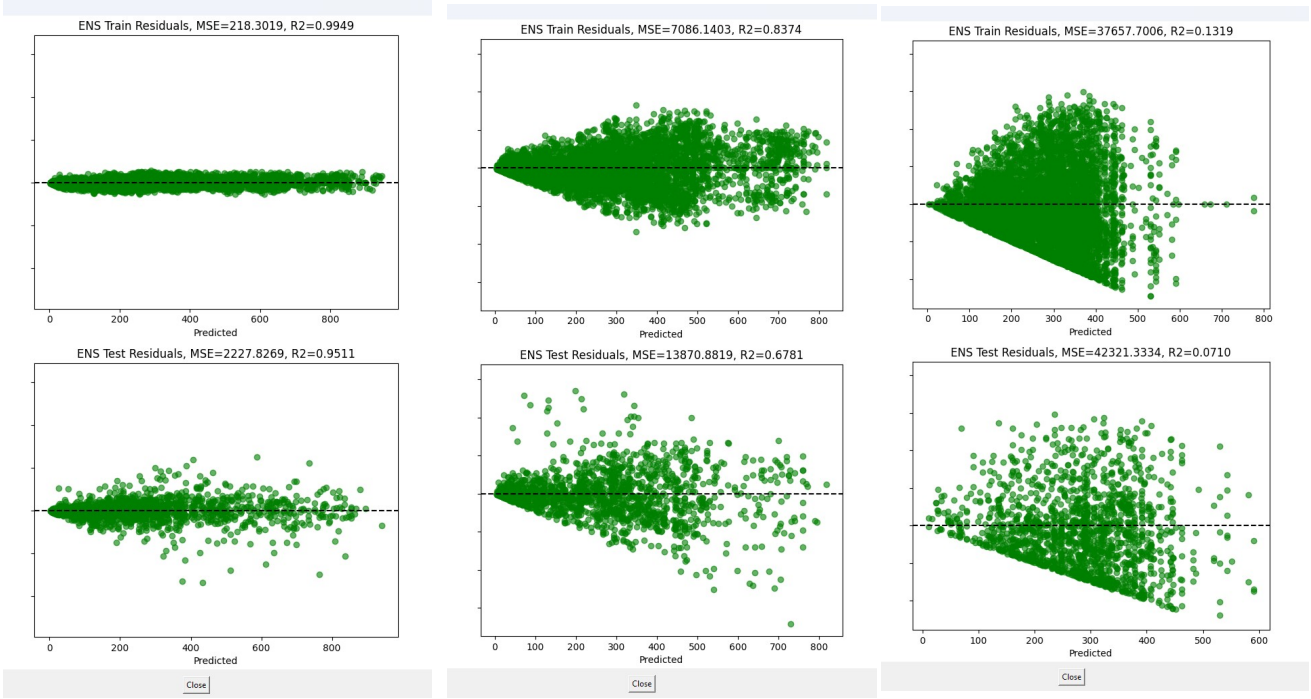
- Vyberte podmnožinu príznakov , vyberte si najúspešnejší model z prvej časti zadania a opäť ho natrénujte pre zmenšenú množinu:
  - podľa korelačnej matice; **1b**

Prepínač **do_42=0**, **do_bivar=1** je teraz dobrý na získanie korelácií medzi všetkými stĺpcami, pričom sú samozrejme cyklické ordinály onehot zakódované na vynútenie striktnej numerickosti každého stĺpca, ktorú si korelačná matica vyžaduje.



Vidíme, že **temperature**, **humidity**, **hour** sú vysoko korelované s cieľom.

do_3=1, **KEPT_X_COLS = ['temperature','humidity','hour'],** pod if do_3 dať najlepší model **_3(x_train, x_test, y_train, y_test, 8, 1, 12, 45, 340, 15)**, vystriedať KEPT_X_COLS s ['weather','windspeed','month']



zľava doprava porovnávame najlepší model so všetkými príznakmi; najlepší model s 3 vynikajúce korelujúcimi príznakmi; najlepší model s 3 zle, ale nie 0-vo korelujúcimi príznakmi

Príznak je lepší, keď je dobre korelovaný s cieľom<=testovacie R2=0.678 vs. 0.071

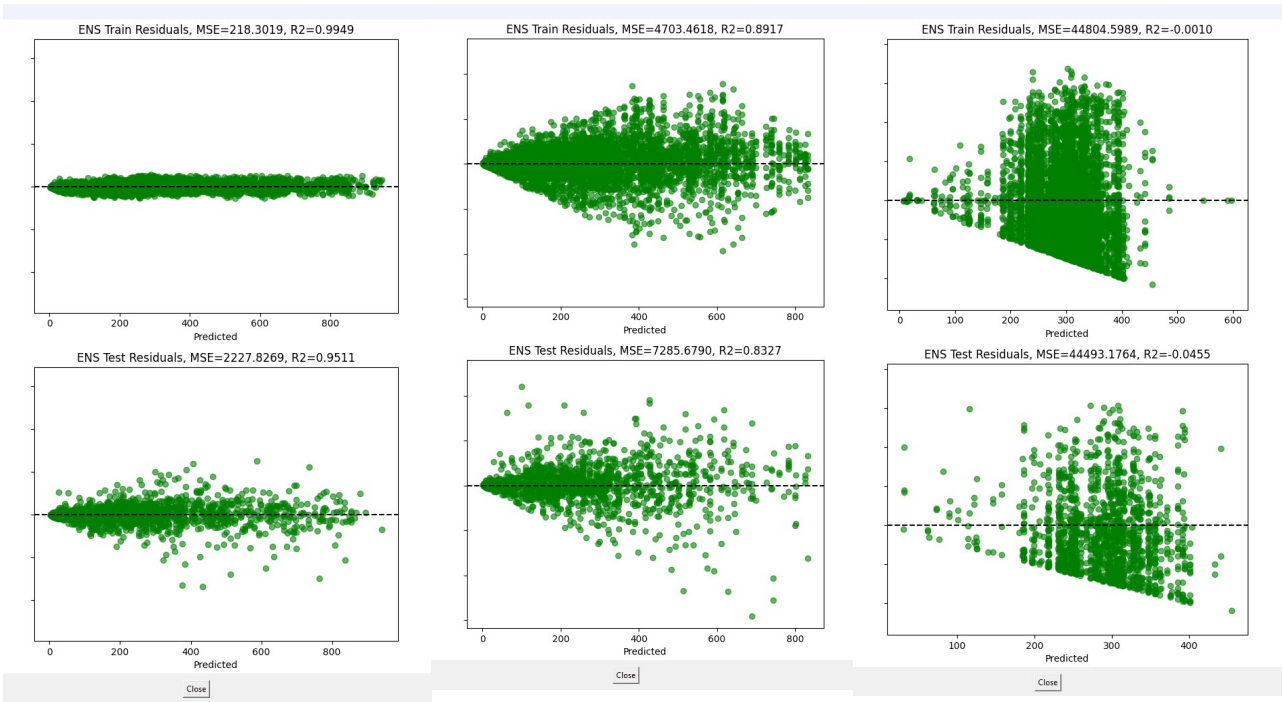
5.2 Víťazný model rieši množinu iba s kvalitnými príznakmi podľa feature importance + metriky

– podľa dôležitosti príznakov z ensemble modelu; **1b**

do_3=1, **KEPT_X_COLS = [],** pod if do_3 dať najlepší model **_3(x_train, x_test, y_train, y_test, 8, 1, 12, 45, 340, 15)** a všimnime si výpis, ktorý _3 píše k ens vždy, kde dobre vidíme, ktoré príznaky boli najnápomocnejšie pri boostingu:

```
0.0 Eda mapped : date->year,month,day; np.nan-> nic , words->numbers; cols kept:
['instant', 'year', 'month', 'day', 'hour', 'holiday', 'weekday', 'workingday',
'weather', 'temperature', 'humidity', 'windspeed', 'count']
1.1 Culled id col instant : (ENTIRE COL DELETED)
1.2 Culled nulls under holiday : 11x nic rows removed
1.3 Culled duplicates : 7x duplicated rows removed
1.4 Culled outliers under temperature :  0 x rows removed
1.4 Culled outliers under humidity :  22 x rows removed
1.4 Culled outliers under windspeed :  0 x rows removed
_3 reports ens feature importances:
year: 0.0000
month: 0.0546
day: 0.0402
hour: 0.4608
holiday: 0.0059
weekday: 0.0259
workingday: 0.1081
weather: 0.0300
temperature: 0.1848
humidity: 0.0693
windspeed: 0.0203
OK
>>>
```

Teraz už vieme, že trénovanie sa bude líšiť v **KEPT_X_COLS = [], KEPT_X_COLS = ['hour','temperature','workingday'] a KEPT_X_COLS=['holiday','weekday','windspeed']**



zľava doprava porovnávame najlepší model so všetkými príznakmi; najlepší model s 3 vynikajúco nápomocnými boostingu príznakmi; najlepší model s 3 zle, ale nie 0-vo nápomocnými boostingu príznakmi

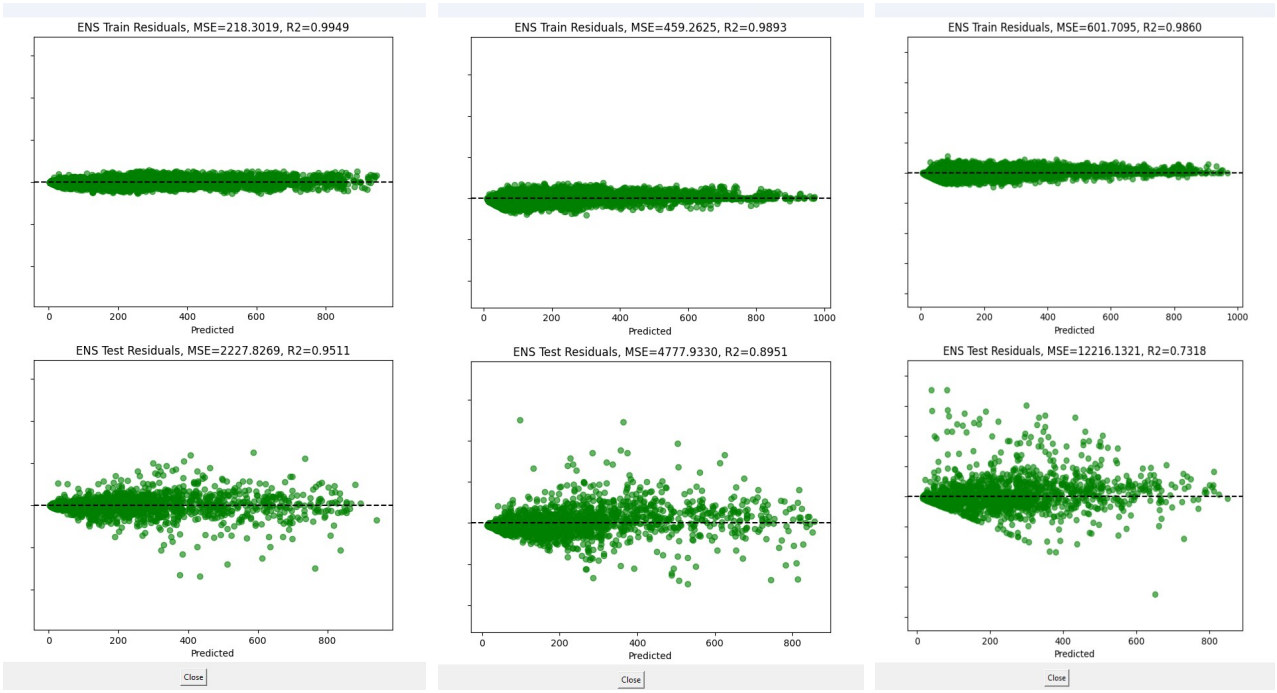
Príznak je lepší, keď je informatívnejší, čo ocení boosting<=testovacie R2=0.832 vs. -0.045

5.3 Víťazný model rieši množinu iba s kvalitnými príznakmi podľa PCA float 0<n_c<1

– podľa variancie pomocou PCA (zvoľte si hodnotu variancie, nie počet príznakov - t.j. nie 3). **1b**

Výsledky porovnajte medzi sebou aj s pôvodným trénovaním pomocou MSE (príp. RMSE), R2 a reziduálov.

do_3=0, do_42=1, **KEPT_X_COLS = [],** vo funkcii _42 zmeniť riadok pca_3d = PCA(n_components=3, random_state=42) na **pca_3d = PCA(n_components=x, random_state=42)**, s tým, že prezrieme _3 normál, x=0.8, x=0.6



zľava doprava porovnávame najlepší model so všetkými príznakmi; najlepší model s príznakmi popisujúcimi 80% variancie; najlepší model s príznakmi popisujúcimi 60% variancie

Aj keď príznaky popisujú menšie množstvo variancie ako celok, ak jednotlivo veľké, tak to nie je tak problematické pre boosting<=testovacie R2=0.895 vs. 0.731

Záver

Práca bola do menšej miery inšpirovaná kódom a testovacími hodnotami skriptov **cvicenie_4.py, ensemble.py** a **svm.py**, ktoré boli prevzaté, autormi sú učitelia predmetu I-SUNS v 2025/2026 na FEI STU BA.

Použitie AI

V časti data cleaningu bolo AI ChatGPT použité len na jednoduché až stredne náročné hľadanie syntaxe a jednoduché až stredne náročné opravy kódu. AI bolo použité vo väčšej miere na generáciu funkcie **errvis** a a v menšej miere na generáciu **_3dvis**. V práci bolo použité AI v menšom meradle na generáciu a opravu kódu súvisiaceho s jednoduchšími EDA grafmi na konci kódu.